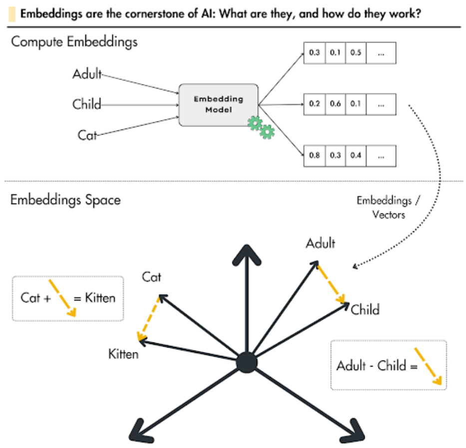
Technical concepts

Last amended: 25th June, 2026
My folder: C:\Users\ashok\OneDrive\Documents\llm
GitHub: https://github.com/harnalashok/LLMs/upload/main/install_ai_tools/misc

Contents

What are embeddings in AI

Embeddings are a list of numbers (called vectors) that are numerical representation of words. Think of embeddings as **GPS coordinates for abstract concepts**. On a normal map, nearby coordinates mean two places are physically close. In an embedding space, nearby coordinates mean two concepts (such as *apple* and *fruit* are *ideally* close).



S

Figure 1: Vectors for (cat, kitten) would be very different from vectors for (adult, child).

Think of embeddings as **GPS coordinates for abstract concepts**. On a normal map, nearby coordinates mean two places are physically close. In an embedding space, nearby coordinates mean two concepts are *ideally* close.

When data passes through an embedding model, it is converted into an array of numbers or **vector** containing hundreds or thousands of decimal numbers (dimensions). Each decimal number in the vector (also called dimension) tracks an abstract feature or relationship learned by the AI during training. Here is a made-up example:

Vector for King: [0.99,0.95,0.7]
Vector for Queen: [0.99,0.95,0.7]
Vector for Apple: [0.99,0.95,0.7]

The numbers in each of the above three vectors may track *Royalty*, *Masculinity* and *Age* respectively. Thus:

Word	Dimension 1: Royalty	Dimension 2: Masculinity	Dimension 3: Age
King	High (0.99)	High (0.95)	Adult (0.7)
Queen	High (0.99)	Low (0.05)	Adult (0.7)
Apple	None (0.00)	Neutral (0.50)	N/A (0.0)

Figure 2: Each number in the array of numbers (vector) indicates some value assigned to some feature of the word.

Embedding model vectors may <u>contain hundreds or thousands of decimal numbers (dimensions).</u> The larger the vector space, more the semantic clarity between words but would require more computational resources to process data. Smaller the vector space, less the computational power required but then less also would be semantic similarity.

Sentence Embedding

Sentence embedding is the process of converting an entire sentence into a single numerical vector that captures its overall meaning and context.

Unlike older methods that look at sentences word-by-word, sentence embeddings analyse the structure, order, and intent of the full phrase. This allows computers to compare different sentences based on their actual ideas rather than the exact words used.

If you embed words individually, the sentences *"The dog bit the man"* and *"The man bit the dog"* look identical because they contain the exact same words.

Sentence embedding models solve this by looking at the phrase holistically. It analyses the structure, order, and intent of the full phrase. They recognize that:

- **Different words, same meaning:** *"How is the weather today?"* and *"Is it raining outside right now?"* will have vectors placed very close together in mathematical space.
- **Same words, different meaning:** *"The bank of the river"* and *"The bank changed my credit limit"* will be placed far apart because the context of "bank" is entirely different.

What are tokens in GPT or Claude?

Tokens are the basic units that commercial models use for pricing. Roughly speaking, 1 token ≈ 4 characters of English text, or about ¾ of a word. So "100 tokens" is approximately 75 words. Simple punctuation is usually its own token. Examples:

- "Hello!" → ~2 tokens
- "The quick brown fox" → ~4 tokens
- A typical paragraph → ~100–150 tokens

Costs are based on two components:

- **Input tokens** — everything you send to Claude/GPT: your message, any system prompt, conversation history, and attached documents.
- **Output tokens** — everything model generates in response.

Output tokens are typically more expensive than input tokens.

What counts toward your token usage?

- Your typed message
- The full conversation history (in multi-turn chats)
- System prompts (set by the operator/platform)
- Uploaded files or documents (their text content)
- Tool results (e.g., web search results passed back to the model)
- Longer conversations accumulate more input tokens over time, since prior messages are re-sent each turn.

Here's the key insight:

Unit	Approximate size
1 token	~4 characters
1 average word	~5 characters (including the space after it)
So 1 word	~1.3 tokens
Therefore 100 tokens	~75-80 words

What is an API?

An **API**, or **Application Programming Interface**, is **a software intermediary that allows two different applications to interact and share data with one another**. Think of it as a digital bridge or a translator that lets separate programs "speak" the same language without needing to know how each other's internal code is written. Wikipedia +3

The Restaurant Analogy

To understand how an API works, imagine you are a customer dining at a restaurant: YouTube · Postman +1

- **The Customer (The Client):** You sit at the table and want to order food. In technology, this is an app or website.
- **The Kitchen (The Server):** The kitchen prepares the food. This represents the backend database or system holding the data.
- **The Waiter (The API):** You don't walk into the kitchen to cook. Instead, you read the menu and give your order to the waiter. The waiter carries your request to the kitchen, retrieves the cooked food, and delivers it back to you. GitHub +3

How Web APIs Work

Most modern web communication relies on a structured **Request-and-Response** loop: YouTube · Exponent +1

1. **The Call:** Your application sends an **API request** over the internet using an HTTP protocol. GitHub +1
2. **The Endpoint:** The request travels to a specific URL path, known as an **endpoint**, which dictates what data is being asked for. YouTube · CodeWithChris +1
3. **The Method:** The request uses strict commands such as **GET** (to fetch data) or **POST** (to submit new data). GitHub +1
4. **The Security Check:** The system verifies the app's identity using a digital password called an **API key** to ensure the request is authorized. YouTube · CodeWithChris +1
5. **The Payload:** The server processes the task and sends back the data, typically formatted in a clean, machine-readable text format called **JSON**. YouTube · Exponent +1

What is a webhook?

The Doorbell Analogy
Think of a **traditional API** like a customer repeatedly calling a restaurant to ask, "Is my takeout order ready yet?". A **webhook** is like leaving your phone number with the restaurant so *they* can automatically text you the moment your food is ready.

Here, replace the 'customer' with a *web-server (say, App-A)*, 'restaurant' with another *web-server (say App-B)* and 'calling' with making an API request to 'App-B'; your 'phone number' with your 'URL or your API. Thus, instead of 'App-A' sending an API request to 'App-B' for some information, 'App-B' makes an API request to 'App-A' informing him of an event.

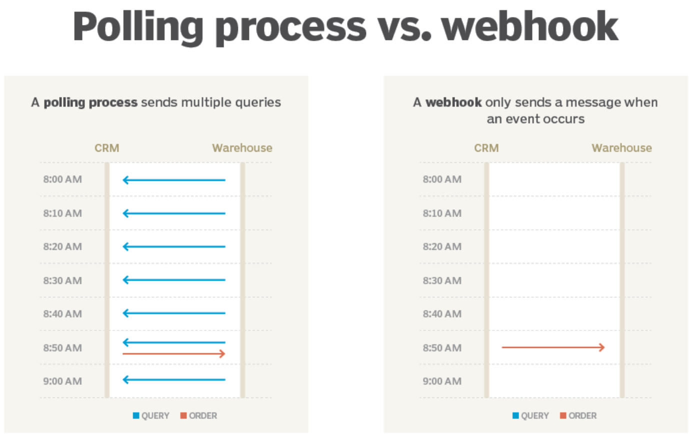


Figure 3: Instead of CRM, repeatedly sending a message to know if the customer order is dispatched, Warehouse, informs CRM as soon as customer order is dispatched.

API vs Webhook

APIs are *pull-based*: Your application (App-A) actively asks for data on-demand (you request it).

Webhooks are *push-based*: The source application (App-B) automatically sends the data to your application the moment something occurs

Webhook is an "Event-based HTTP callback trigger"—a long name. Name, "Webhook" is punchy, precise, and easily turned into a verb (e.g., *"We can just webhook that data over"*). Webhook uses using standard web protocols (HTTP): POST/GET.

Ollama parameters

Ollama parameters are customizable settings that control how a large language model (LLM) generates text, balancing its creativity, predictability, memory, and formatting. They can be adjusted in the terminal or permanently saved in a **Modelfile.**

Key Generation Parameters

These parameters control the AI's "thinking" process and output style.

- `temperature` **(Default: `0.8`):** Controls the creativity and randomness of the output.
  - **Low (e.g., `0.1` - `0.3`):** Makes the output more factual, logical, and predictable. Best for coding, math, or data extraction.
  - **High (e.g., `0.8` - `1.0`):** Makes the output more creative, diverse, and unpredictable. Best for brainstorming or writing stories. Medium · Laurent Kubaski +4
- `top_p` **(Default: `0.9`):** An alternative way to control randomness, often called "nucleus sampling". It tells the model to only consider the pool of words whose combined probabilities add up to a certain percentage (e.g., 0.9 means the model considers the top 90% most likely words, cutting off bizarre, low-probability words). YouTube · Matt Williams +1
- `top_k` **(Default: `40`):** Limits the model to only choose from its top K most likely next words. A lower number (e.g., `10`) keeps the output focused; a higher number (e.g., `50`) allows for more diverse vocabulary. Ollama +2
- `seed` **:** Sets a specific starting point for the model's random number generator. If you use the same seed, prompt, and parameters, you will get the exact same response every time. YouTube · Matt Williams +2
- `repeat_penalty` **(Default: `1.1`):** Penalizes the model for repeating the same words or phrases too often. Higher numbers result in more diverse answers. Ollama +1
- `repeat_last_n` **(Default: `64`):** Controls how far back the model looks in its own output to check for repetition. Ollama +1

Here is a graphical representation of the effect of *temperature* vs *top_p*. Think of text generation as a funnel: **top_p cuts off** the unlikely choices to shape the pool, while **temperature reshapes the probabilities** of the remaining choices.

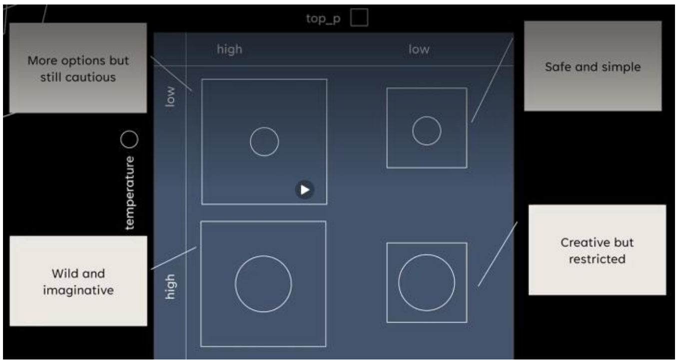


Figure 4: top_p vs temperature interaction

- Low top_p and temperature
  - Factual Q&As
  - Legal documents
  - Code generation
- High top_p, low temperature
  - Executive summaries
  - Sensitive content
- Low top_p, high temperature
  - Casual chatbots
  - Humorous tone
- High top_p and temperature
  - Stories
  - Poetry
  - Brainstorming

Figure 5: top_p vs temperature: when to use

Here is a video that explains very nicely the twin-concepts of *top_p* and *temperature*.

Figure 6: Excellent video on top_p vs temperature

Here are the four scenarios of interaction of these two parameters.

Four Common Interaction Scenarios

1. Low Temp + Low Top_p (The Strict Academic)

- **Settings**: `temperature 0.2`, `top_p 0.5`
- **Interaction**: `top_p` eliminates everything except the top handful of highly predictable words. Then, `temperature` makes the highest-scoring word look even more dominant.
- **Result**: Highly repetitive, extremely safe, and deterministic text. This combo is perfect for code syntax, math, and data extraction where mistakes are not allowed.

2. High Temp + High Top_p (The Wild Brainstormer)

- **Settings**: `temperature 1.2`, `top_p 0.95`
- **Interaction**: `top_p` leaves the door wide open, keeping almost every obscure and rare word in play. Then, `temperature` flattens the odds, allowing those rare, chaotic words to frequently win the selection lottery.
- **Result**: Highly creative, unpredictable, and poetic responses. However, it significantly increases the risk of hallucinating facts or generating grammatical gibberish.

3. High Temp + Low Top_p (The Controlled Novelty)

- **Settings**: `temperature 1.2`, `top_p 0.4`
- **Interaction**: `top_p` strictly limits the selection to only the top 40% most logical words. However, because `temperature` is high, the model will aggressively cycle between those few valid options rather than just picking the #1 choice.
- **Result**: Vivid, lively writing that still adheres strictly to the topic without drifting off into nonsense.

Best Practices for Tuning

- **Adjust one at a time**: If you tweak both simultaneously, it is difficult to tell which parameter caused a shift in tone.
- **Keep one at default**: The industry standard recommendation is to leave `top_p` at its default value (`0.9` or `0.95`) and use `temperature` as your primary dial for creativity.
- **The Coding Override**: If you need absolute precision, set `temperature` to `0.0`. This forces the model to always pick the absolute highest probability token, rendering `top_p` completely irrelevant.

Custom Ollama Models

For Modelfile, please see this [GitHub link]
Opewebui for creating online modelfile [link is here]

Custom Ollama models are **personalized versions of existing AI models (like Llama 3 or Mistral) modified to behave in specific ways**. Created using a simple script called a **Modelfile**, they allow you to set custom system prompts, adjust behavior parameters, or load your own fine-tuned weights—all running locally on your hardware. YouTube · Douglas Starnes +1

Key Ways to Customize Models

- **System Prompts & Personas:** Change the core identity of the model. For instance, you can turn a base model into a stern code reviewer, an expert copywriter, or a specific character. YouTube · The Neural Maze +2
- **Parameters:** Adjust the settings for how the model generates responses. You can increase the `temperature` for more creative responses, adjust `num_ctx` to give it a larger memory/context window, or set `stop` sequences. YouTube · The Neural Maze +2
- **Templates:** Dictate exactly how prompts are structured before being fed into the AI. YouTube · The Neural Maze +1
- **Custom Weights (GGUF):** Import your own fine-tuned model weights or specialized models (like those from Hugging Face) to give the AI entirely new knowledge domains. Ollama +1

How They Work: The Modelfile

A Modelfile is essentially a blueprint that tells Ollama how to construct your custom assistant, working much like a Dockerfile. Once written, you use the `ollama create` command to build it into an independent model that you can call upon anytime. Ollama +3

How They Work: The Modelfile

A Modelfile is essentially a blueprint that tells Ollama how to construct your custom assistant, working much like a Dockerfile. Once written, you use the `ollama create` command to build it into an independent model that you can call upon anytime. Ollama +3

Here is a quick example of a Modelfile built to make a code-reviewing assistant:

text

```
# Start from a capable base model
FROM llama3.2

# Set the system prompt that defines behavior
SYSTEM """
You are an expert code reviewer. When analyzing code:
1. Check for security vulnerabilities first.
2. Suggest improvements for readability.
"""

# Adjust inference parameters
PARAMETER temperature 0.3
PARAMETER num_ctx 8192
```

Use code with caution.

See the below video on creating custom ollama models.

Figure 7: How to create custom ollama models